

IoT Engineering Curriculum Workbook

COMPREHENSIVE PROJECT WORKFLOW & LAB EXECUTION MANUAL

Overview: This structured guide maps out a complete educational pathway for teaching students embedded system controls using your specialized file library. The workspace contains 13 specific implementation assets ranging from basic physical hardware peripheral interaction up to enterprise cloud telemetry and broker integration.

By following the specific workflow stages detailed below, students transition from local firmware primitives up through web models, local servers, cloud bridges, and lightweight message streaming protocols.

Workspace Code Asset Catalog

The codebase consists of three integrated domains: Microcontroller Firmware (.ino), Web Frontends (.html), and Local Microservice Backends (.py). Below is the taxonomy matching your screenshot:

Code Asset Filename	Environment Type	Primary Educational Objective
<code>digital_wite.ino</code>	ESP32 Core / C++	Basic digital logic output; driving internal/external pins HIGH/LOW.
<code>digital_read_and_write.ino</code>	ESP32 Core / C++	Polling physical input flags (buttons) to instantly manipulate output pins.
<code>analog_read_analog_write.ino</code>	ESP32 Core / C++	Sampling multi-level analog voltage (ADC) and duty-cycle generation (PWM).
<code>anlog_read_digital_write.ino</code>	ESP32 Core / C++	Mapping analog threshold ranges to discrete true/false digital state actions.
<code>contorl_webpage_for_esp_server.html</code>	Client Browser / JS	Detached browser control face managing remote targets using text input IPs.
<code>ESP_Server_to_control_the_led.ino</code>	ESP32 Core / C++	Setting up a local, un-styled micro HTTP routing target on port 80.
<code>control_webpage_esp_server_inbuilt.ino</code>	ESP32 Core / C++	Embedding native HTML layouts inside raw Flash memory to host self-contained servers.
<code>HTTP_server_to_control_the_led(gist).ino</code>	ESP32 Core / C++	Global cloud fetching model utilizing remote key-value text lines over HTTPS.

Code Asset Filename	Environment Type	Primary Educational Objective
<code>HTTP_server_to_control_the_led(loacar_server) copy.ino</code>	ESP32 Core / C++	Network-isolated local text polling aimed directly at a nearby notebook proxy.
<code>http_data_recive_in_local_server.py</code>	Python / Flask App	Configuring a local workstation listener webserver to receive incoming POST payloads.
<code>HTTP_data_send_to_local_server.ino</code>	ESP32 Core / C++	Active telemetry parsing; packing chip readings and posting them to a local script.
<code>control_webpage_for_mqtt_server.html</code>	Web Application	Client GUI configured to leverage web-sockets for cloud-broker messaging.
<code>MQTT_Server_to_control_the_led.ino</code>	ESP32 Core / C++	Subscribing to distributed pub-sub structures for event-driven processing.

Integrated Learning & Implementation Workflow

To teach or build this system effectively, the files must not be treated in isolation. Execute the following sequential curriculum progression:

PHASE A: LOCAL EMBEDDED PRIMITIVES & FOUNDATIONS

Step 1: Output Essentials

`digital_wite.ino`



Step 2: Interactive IO

`digital_read_and_write.ino`



Step 3: Continuous Signals

`analog_read_analog_write.ino`
`anlog_read_digital_write.ino`

1 Hardware IO Mastery (Prerequisite Track)

Before launching web stacks, ensure students map the spatial architecture of their microcontrollers. This block establishes safe operating voltages, current limitations, and physical circuit builds.

- Deploy `digital_wite.ino` to demonstrate basic high/low voltage transitions (e.g., blinking test lights).
- Layer on `digital_read_and_write.ino` to introduce interrupt polling paradigms—detecting physical mechanical switches safely.
- Transition to mixed wave control via `analog_read_analog_write.ino` and `anlog_read_digital_write.ino`. This teaches students how to process analog variations (such as temperature, distance, or variable dial resistances) and map those numbers to variable brightness or physical safety logic.

IMPORTANT NOTE: VOLTAGE SAFETY BOUNDARIES

The ESP32 processes signals strictly at **3.3V maximum logic level**. Feeding a raw 5V sensor line directly into an analog-to-digital converter pin will degrade internal chip pathways permanently. Use simple voltage divider resistors when linking traditional industrial legacy components.

PHASE B: LOCAL NETWORK INBOUND SERVER ARCHITECTURE

Step 4: Separated UI

`contorl_webpage_for_esp_server.html`



Step 5: Router Base Target

`ESP_Server_to_control_the_led.ino`



Step 6: Self-Contained Edge

`control_webpage_esp_server_inbuilt.ino`

2 Local Network Client-to-Server Controls

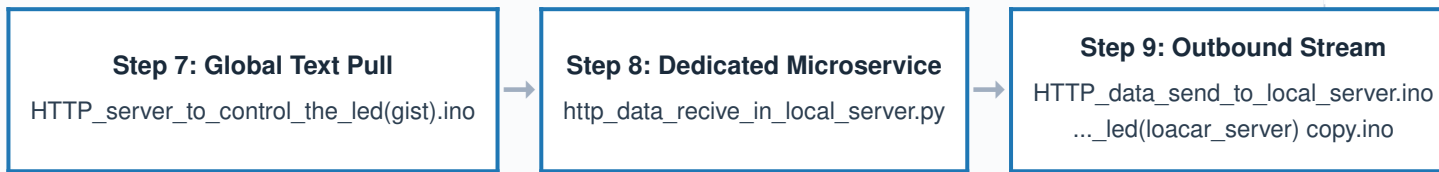
This phase demonstrates the concept of the **Local Webserver**, showing how an embedded device on a standard home router can process incoming browser packets without needing an active internet subscription.

- Flash the device with `ESP_Server_to_control_the_led.ino`. Instruct students to view the allocated local IP through their debug terminals.
- Launch `control_webpage_for_esp_server.html` locally on any workstation. Input the chip's IP into the dashboard box to toggle live hardware indicators using un-routed asynchronous JavaScript commands.
- Evolve to the optimal modern design using `control_webpage_esp_server_inbuilt.ino`. This bundles the entire user interface inside the ESP32's flash memory. Typing the IP address directly into any smartphone browser on the network downloads the controls dynamically from the hardware itself.

PRO-TIP: ASYNCHRONOUS NETWORK PERFORMANCE

By moving away from standard full-page web updates to headless background fetch controls (`mode: 'no-cors'`), you eliminate severe rendering lag on the microcontroller, making hardware updates feel instantaneous.

PHASE C: DISTRIBUTED CLIENT MODELS & DATA COLLECTION (TELEMETRY)



3 HTTP Client Polling & Active Data Collection

Here, the execution logic flips. The ESP32 is no longer a server waiting for connections. It now acts as an active web client, pulling remote configuration tables down and pushing hardware readings up.

- Deploy `HTTP_server_to_control_the_led(gist).ino` to read states over the global internet from a public text file placeholder. Students can modify a simple cloud text file to watch their devices update across different subnets.
- Boot up `http_data_recive_in_local_server.py` on a computer to initiate a network listener endpoint using Flask.
- Run `HTTP_data_send_to_local_server.ino` and `HTTP_server_to_control_the_led(loacar_server) copy.ino`. The chip samples local environment pins and regularly posts structured data records right to your computer's terminal.

CRITICAL SYSTEM RISK: POLLING OVERHEAD & SERVER LOAD

Continuous HTTP polling opens and tears down massive TCP stacks every single cycle. If a classroom of thirty students sets an unrestricted loop delay (`delay(10)`) targeting a single central local script, the massive packet collision overhead can crash the network interface completely. Maintain a minimum check delay of 1000ms.

PHASE D: EVENT-DRIVEN ENTERPRISE ARCHITECTURE (MQTT MESSAGING)



4 Event-Driven Streaming Protocols (MQTT)

The final educational milestone replaces inefficient HTTP polling with real-time, event-driven publish/subscribe pipelines. This introduces industry-standard, lightweight messaging architectures.

- Flash the device with `MQTT_Server_to_control_the_led.ino`, configuring it to listen to a specific broker message topic string (e.g., `"workshop/led/switch"`).
- Launch the companion asset `control_webpage_for_mqtt_server.html`. When a user interacts with the interface, it publishes a tiny raw text payload directly to the central cloud broker. The broker immediately pushes that message to all listening microcontrollers over a single long-lived connection.

ARCHITECTURE SUMMARY: WHY MQTT OUTPERFORMS HTTP

HTTP requires the chip to ask for status updates constantly, consuming substantial processing time and network capacity. MQTT holds open a thin, lightweight background data pipeline. The hardware remains completely silent until a configuration change is published, minimizing data traffic and eliminating action lag entirely.